

面向对象程序设计 第1次作业

第1题

1.5 第17题

17. 下面哪些是合法的 C++ 标识符?

`extern, _book, Car, car_1, calr, lcar, friend, car1_Car, Car_Type, No.1, 123`

合法标识符: `_book, Car, car_1, calr, car1_Car, Car_Type`

不合法标识符及原因:

- `extern`: 属于 C++ 关键字, 不能用作标识符。
- `lcar`: 标识符不能以数字开头。
- `friend`: 属于 C++ 关键字, 不能用作标识符。
- `No.1`: 包含非法字符 `.` (点号)。标识符只能包含字母、数字和下划线。
- `123`: 全数字组成且以数字开头, 不符合命名规则。

第2题

2.7 第7题

7. 计算下面表达式的值 (x 、 y 和 z 是三个 `int` 型变量, 初值分别为 1、2 和 3):

(1) $x + 3/y - 2 * z$

(2) $x > 0 \ || \ y < 2 \ \&\& \ z > 1$

(3) $z > y > x$

(1) $x + 3/y - 2 * z$

• 计算步骤:

1. 处理除法: $3 / y$ 即 $3 / 2$ 。由于是 `int` 型运算, 结果为 1 (舍去小数)。
2. 处理乘法: $2 * z$ 即 $2 * 3$, 结果为 6。

3. 执行加减: `x + 1 - 6` 即 `1 + 1 - 6`。

- 计算结果: -4

(2) `x > 0 || y < 2 && z > 1`

- 计算步骤:

1. 根据优先级, 先计算关系运算符:

- `x > 0` 为 **true** (1)
- `y < 2` 为 **false** (0)
- `z > 1` 为 **true** (1)

2. 处理逻辑运算符 (优先级 `&&` 高于 `||`):

- `y < 2 && z > 1` 为 `false && true`, 结果为 **false** (0)。

3. 最后计算 `||`:

- `x > 0 || false` 即 `true || false`。

- **短路求值说明:** 在实际运行中, 由于 `x > 0` 已为 **true**, 逻辑或 `||` 会发生短路, 不再计算右侧表达式。

- 计算结果: 1

(3) `z > y > x`

- 计算步骤:

1. 关系运算符具有左结合性, 先计算左侧: `z > y` 即 `3 > 2`, 结果为 **true** (在 C++ 表达式评估中返回数值 1)。

2. 再计算剩余部分: `1 > x` 即 `1 > 1`。

- **注意点:** 这是一个典型的逻辑陷阱, 该表达式并非判断 `y` 是否在 `x` 和 `z` 之间, 而是将前者的布尔结果与后者进行数值比较。

- 计算结果: 0

2.7 第8题

(3) `4 < y < 6`

8. 表达式中的类型转换规则是什么? 计算下面的表达式时如何进行操作数类型转换?

(1) `3/5 * 12.3`

(2) `'a' + 10 * 5.2`

(3) `12U + 3.0F * 24L`

(1) $3/5 * 12.3$

- 转换过程：

1. 首先计算 $3/5$ ：由于 `3` 和 `5` 都是 `int` 型，执行整数除法。结果为 `0` (`int` 型)。
2. 接着计算 $0 * 12.3$ ：`0` 是 `int` 型，`12.3` 是 `double` 型。根据规则，`int` 转换为 `double`。
3. 最终运算： $0.0 * 12.3$ 。

- 计算结果：**0.0**

(2) $'a' + 10 * 5.2$

- 转换过程：

1. 处理乘法 $10 * 5.2$ ：`10` 是 `int` 型，`5.2` 是 `double` 型。`int` 转换为 `double`，计算结果为 `52.0` (`double` 型)。
2. 处理加法 $'a' + 52.0$ ：`'a'` 是 `char` 型 (ASCII 值为 97)，`52.0` 是 `double` 型。`char` 首先提升为 `int`，再提升为 `double`。
3. 最终运算： $97.0 + 52.0$ 。

- 计算结果：**149.0**

(3) $12U + 3.0F * 24L$

- 转换过程：

1. 处理乘法 $3.0F * 24L$ ：`3.0F` 是 `float` 型，`24L` 是 `long` 型。`long` 转换为 `float`。计算结果为 `72.0` (`float` 型)。
2. 处理加法 $12U + 72.0F$ ：`12U` 是 `unsigned int` 型，`72.0F` 是 `float` 型。`unsigned int` 转换为 `float`。
3. 最终运算： $12.0F + 72.0F$ 。

- 计算结果：**84.0**

2.7 第10题

10. 写出下列条件的 C++ 逻辑表达式。

(1) i 能被 j 整除。

(2) ch 为字母字符。

(3) m 为偶数。

(4) n 是小于 100 的奇数。

(5) a 、 b 、 c 构成三角形的三条边。

(6) a 、 b 、 c 不构成等边三角形。

(1) i 能被 j 整除

```
i % j == 0
```

需确保 $j \neq 0$

(2) ch 为字母字符

```
(ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z')
```

(3) m 为偶数

```
m % 2 == 0
```

(4) n 是小于 100 的奇数

```
n < 100 && n % 2 != 0
```

(5) a, b, c 构成三角形的三条边

```
a + b > c && a + c > b && b + c > a
```

(6) a, b, c 不构成等边三角形

```
!(a == b && b == c)
```

第3题

3.8 第7题

7. 假定邮寄包裹的计费标准如下（重量在档次之间时往上靠）：

重量（克）	收费（元）
15	5
30	9
45	12
60	14（每满 1000 千米加收 1 元）
60 以上	15（每满 1000 千米加收 2 元）

编写一个程序，输入包裹重量和邮寄距离，计算并输出收费数额。

1. 代码

```
#include <iostream>
#include <cmath>

using namespace std;

int main() {
    double weight, distance;
    double totalFee = 0;

    cout << "请输入包裹重量（克）和邮寄距离（千米）： ";
    if (!(cin >> weight >> distance)) {
        return 1;
    }
}
```

```

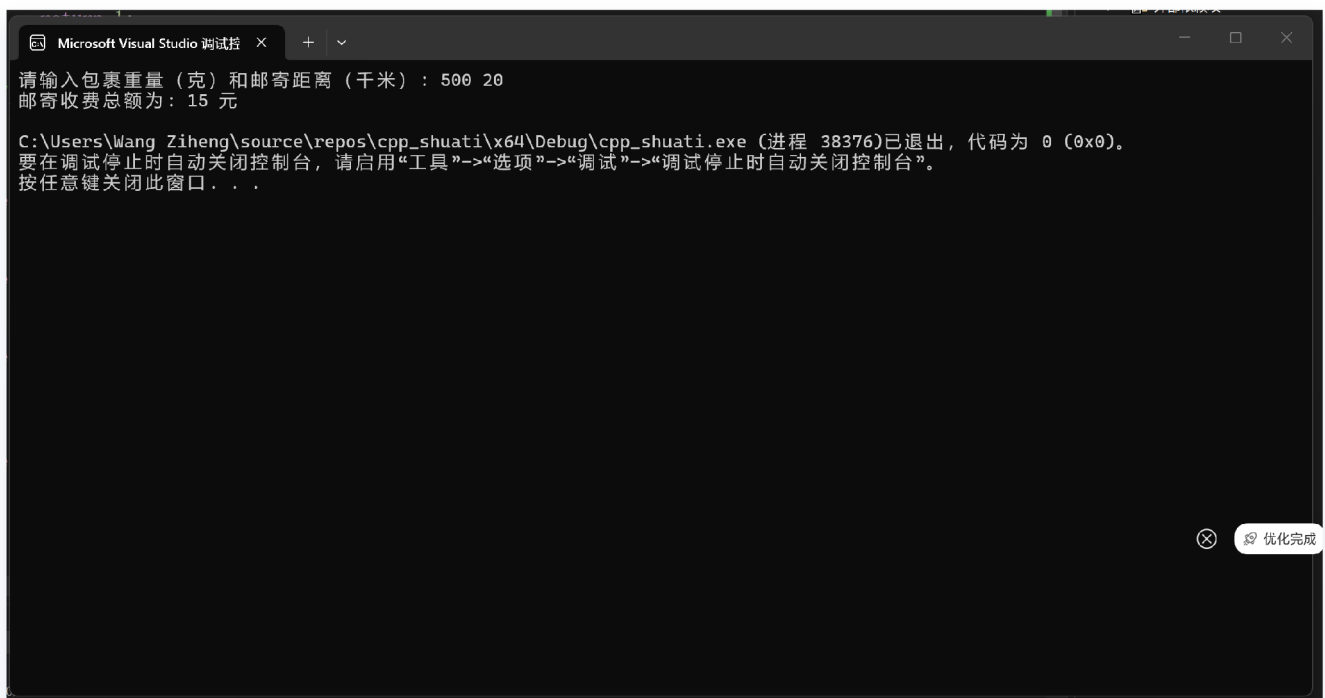
// 根据重量确定基础计费逻辑
if (weight ≤ 15) {
    totalFee = 5;
} else if (weight ≤ 30) {
    totalFee = 9;
} else if (weight ≤ 45) {
    totalFee = 12;
} else if (weight ≤ 60) {
    // 14元基础费 + 每满1000千米加收1元
    totalFee = 14 + floor(distance / 1000) * 1;
} else {
    // 60克以上: 15元基础费 + 每满1000千米加收2元
    totalFee = 15 + floor(distance / 1000) * 2;
}

cout << "邮寄收费总额为: " << totalFee << " 元" << endl;

return 0;
}

```

2.运行截图



3.8 第8题

8. 编写一个程序计算圆周率。可利用公式：

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots$$

第 3 章 程序流程控制

直到最后一项的绝对值小于 10^{-8} 。

1. 代码

```
#include <iostream>
#include <iomanip>
#include <cmath>

using namespace std;

int main() {
    double pi_over_4 = 0.0;
    double term = 1.0;    // 当前项的值
    double denominator = 1.0; // 分母
    int sign = 1;          // 符号位, 正负交替

    // 循环直到当前项的绝对值小于 10^-8
    while (fabs(term) >= 1e-8) {
        term = sign * (1.0 / denominator);
        pi_over_4 += term;
    }
}
```

```

        // 更新下一项的参数
        denominator += 2.0;
        sign = -sign;
    }

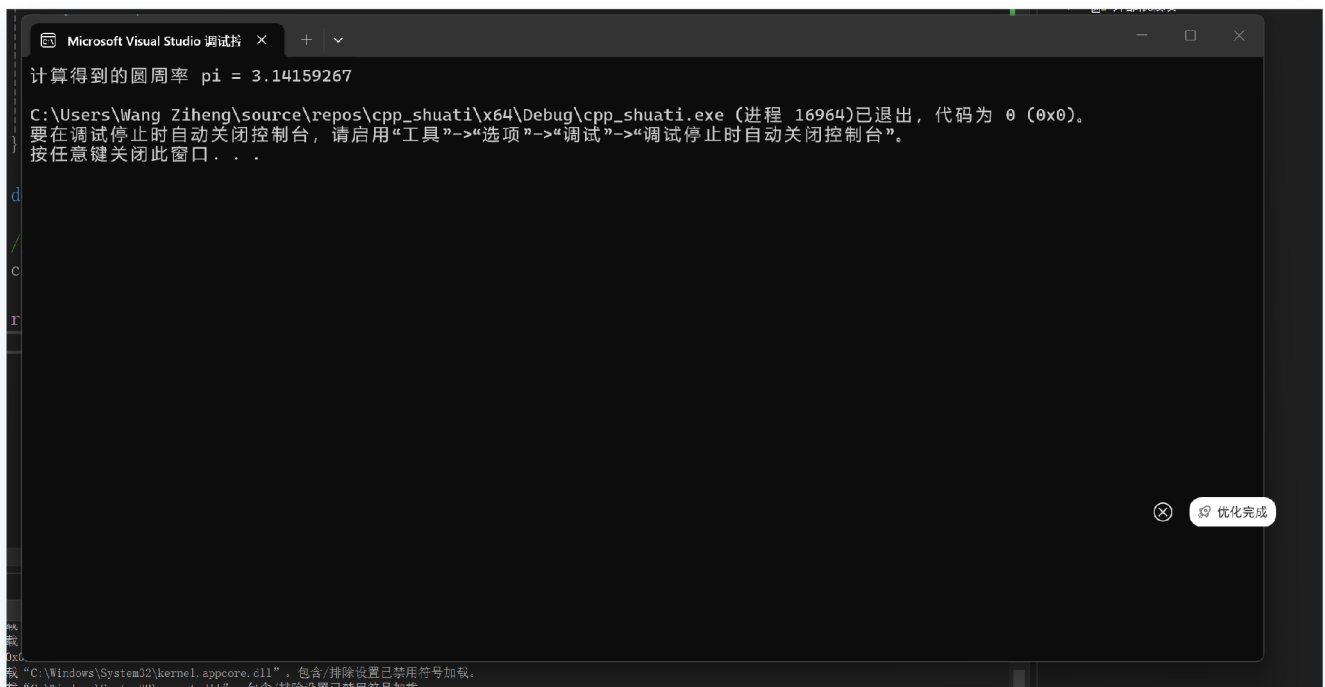
    double pi = pi_over_4 * 4.0;

    // 输出结果，保留高精度以验证计算效果
    cout << "计算得到的圆周率 pi = " << fixed << setprecision(8) << pi << endl;

    return 0;
}

```

2.运行截图



3.8 第11题

11. 现有 1 元、2 元和 5 元的货币（数量不限），请计算购买价值为 n 元的物品，有多少种支付方式（要求输出每一种支付方式）。

1.代码

```
#include <iostream>
```



```

using namespace std;

int main() {
    int n;
    int count = 0;

    cout << "请输入物品价值 n: ";
    if (!(cin >> n)) return 0;

    cout << "所有可能的支付方式如下: " << endl;
    cout << "1元\t2元\t5元" << endl;
    cout << "-----" << endl;

    // 穷举 5 元硬币的数量
    for (int five = 0; five ≤ n / 5; ++five) {
        // 穷举 2 元硬币的数量
        for (int two = 0; two ≤ (n - five * 5) / 2; ++two) {
            // 剩余金额全部由 1 元硬币支付
            int one = n - five * 5 - two * 2;

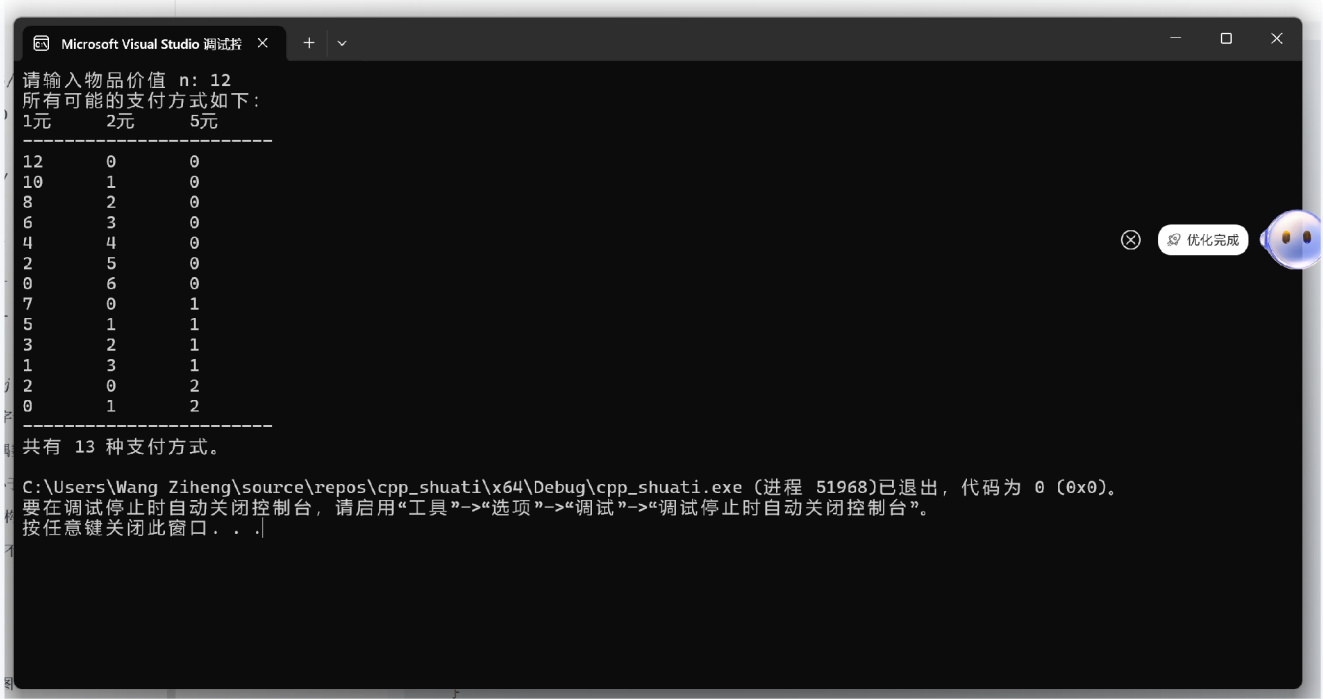
            // 输出当前组合
            cout << one << "\t" << two << "\t" << five << endl;
            count++;
        }
    }

    cout << "-----" << endl;
    cout << "共有 " << count << " 种支付方式。" << endl;

    return 0;
}

```

2.运行截图



第4题

4.8 第12题

12. 写出计算 Ackermann 函数 $ack(m, n)$ 值的递归函数并用它计算 $ack(2, 2)$ 的值。 $ack(m, n)$ 定义如下 ($m \geq 0, n \geq 0$):

$$ack(0, n) = n + 1$$
$$ack(m, 0) = ack(m - 1, 1)$$
$$ack(m, n) = ack(m - 1, ack(m, n - 1)) \quad (m > 0, n > 0)$$

1.代码

```
#include <iostream>

using namespace std;

// Ackermann 递归函数
int ack(int m, int n) {
    if (m == 0) {
        return n + 1;
    } else if (n == 0) {
        return ack(m - 1, 1);
    } else {
        return ack(m - 1, ack(m, n - 1));
    }
}
```

```

        return ack(m - 1, 1);
    } else {
        return ack(m - 1, ack(m, n - 1));
    }
}

int main() {
    int m = 2, n = 2;
    cout << "ack(" << m << ", " << n << ") = " << ack(m, n) << endl;
    return 0;
}

```

2. $ack(2, 2)$ 的手工推导过程

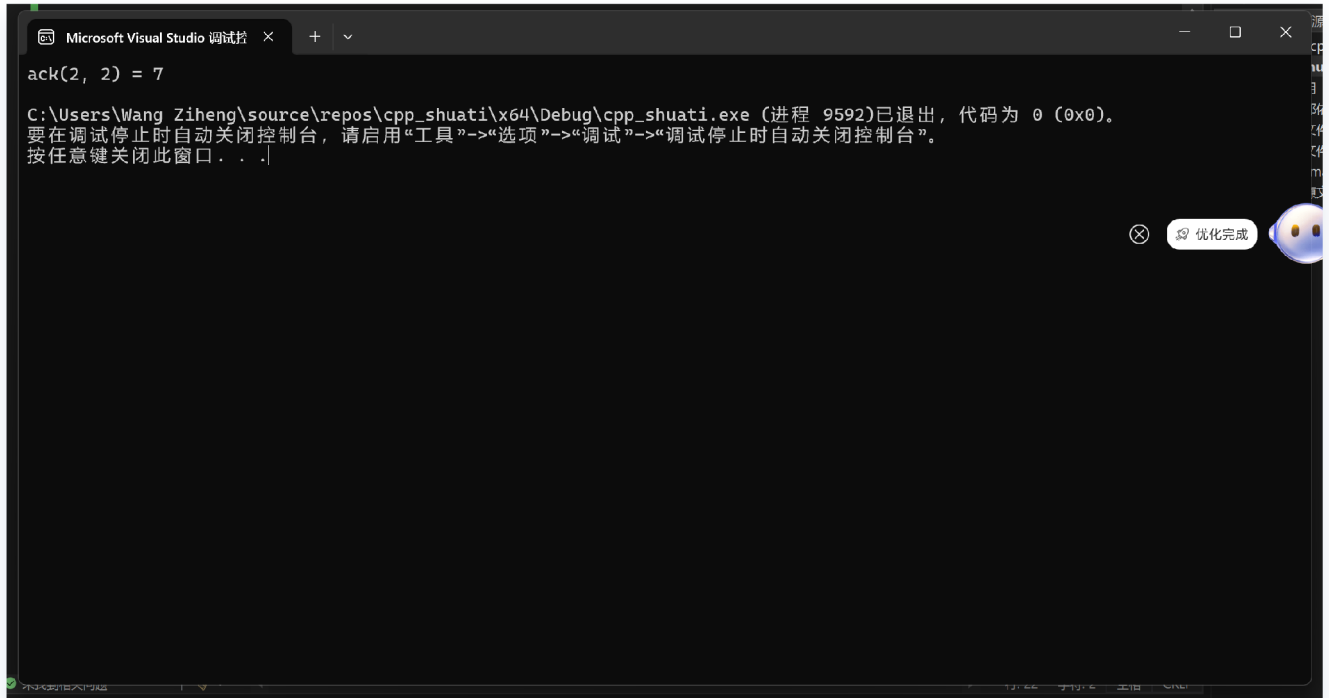
为了计算 $ack(2, 2)$ ，我们需要按照递归定义逐步展开：

- $ack(2, 2) = ack(1, ack(2, 1))$
 - 先计算 $ack(2, 1) = ack(1, ack(2, 0))$
 - 再计算 $ack(2, 0) = ack(1, 1)$
 - $ack(1, 1) = ack(0, ack(1, 0))$
 - $ack(1, 0) = ack(0, 1) = 1 + 1 = 2$
 - 由此得 $ack(1, 1) = ack(0, 2) = 2 + 1 = 3$
 - 回代得 $ack(2, 1) = ack(1, 3)$
 - $ack(1, 3) = ack(0, ack(1, 2))$
 - $ack(1, 2) = ack(0, ack(1, 1)) = ack(0, 3) = 4$
 - 由此得 $ack(1, 3) = ack(0, 4) = 5$
- 回代最终式： $ack(2, 2) = ack(1, 5)$
 - $ack(1, 5) = ack(0, ack(1, 4))$
 - $ack(1, 4) = ack(0, ack(1, 3)) = ack(0, 5) = 6$
 - 由此得 $ack(1, 5) = ack(0, 6) = 7$

计算结果：

$$ack(2, 2) = 7$$

3.运行截图



第5题

5.8 第12题

12. 实现下面的数组元素交换位置函数：

```
void swap(int a[], int m, int n);
```

该函数能够把数组 a 的前 m 个元素与后 n 个元素交换位置，即

交换前： $a_1, a_2, \dots, a_m, a_{m+1}, a_{m+2}, \dots, a_{m+n}$

交换后： $a_{m+1}, a_{m+2}, \dots, a_{m+n}, a_1, a_2, \dots, a_m$

要求：除数组 a 外，不得引入其他数组。

1.代码

```
#include <iostream>

using namespace std;

void reverse(int a[], int left, int right) {
```

```

        while (left < right) {
            int temp = a[left];
            a[left] = a[right];
            a[right] = temp;
            left++;
            right--;
        }
    }

void swap(int a[], int m, int n) {
    reverse(a, 0, m - 1);

    reverse(a, m, m + n - 1);

    reverse(a, 0, m + n - 1);
}

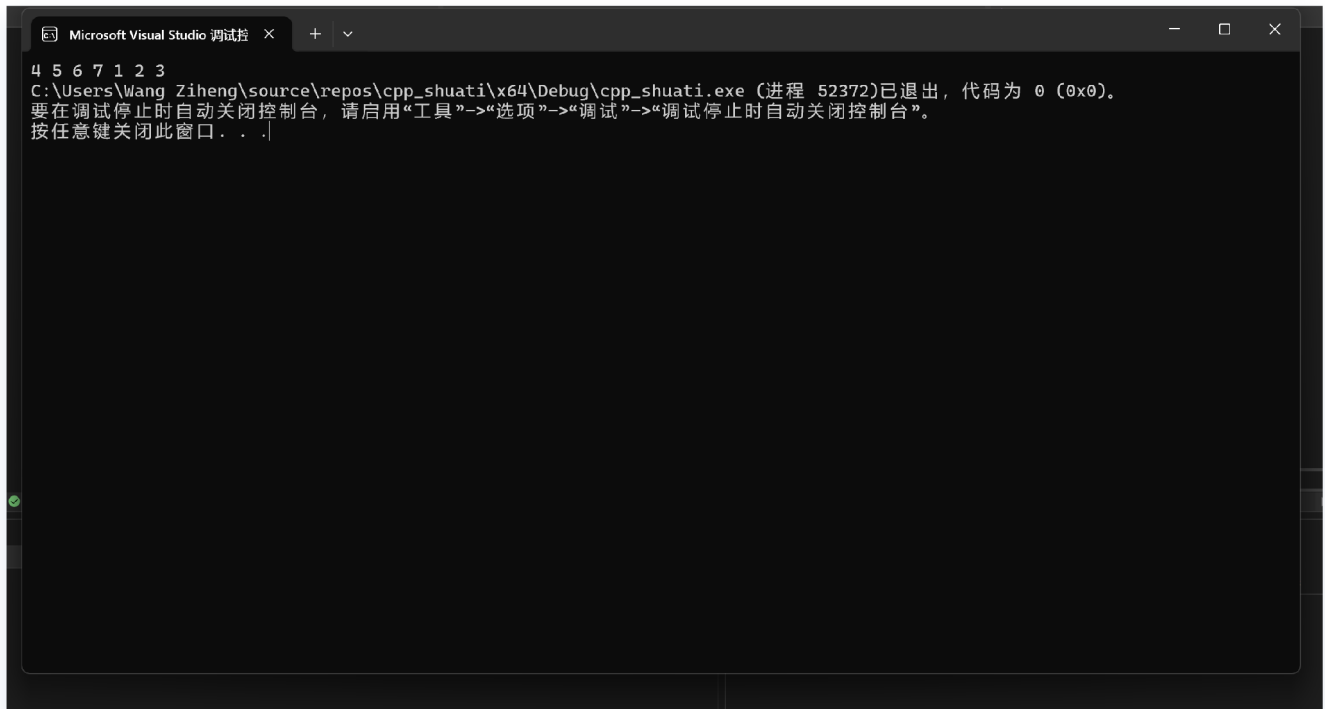
int main() {
    int m = 3, n = 4;
    int a[] = {1, 2, 3, 4, 5, 6, 7}; // 前3个: {1,2,3}, 后4个: {4,5,6,7}

    swap(a, m, n);

    // 输出验证结果
    for (int i = 0; i < m + n; i++) {
        cout << a[i] << " ";
    }
    // 预期输出: 4 5 6 7 1 2 3
    return 0;
}

```

2.运行截图



5.8 第18题

18. 编写一个程序，从键盘输入一批学生的成绩信息，每个学生的成绩信息包括学号、姓名以及 8 门课的成绩。然后按照平均成绩由高到低的顺序输出学生的学号、姓名、8 门课的成绩以及平均成绩。

1.代码

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <iomanip>

using namespace std;

// 定义学生信息结构体
struct Student {
    string id;           // 学号
    string name;         // 姓名
    double scores[8];    // 8门课成绩
    double average;      // 平均分
};
```

```

// 计算平均分的成员函数
void calculateAverage() {
    double sum = 0;
    for (int i = 0; i < 8; ++i) {
        sum += scores[i];
    }
    average = sum / 8.0;
}

};

// 比较函数: 用于 sort 降序排列
bool compareStudents(const Student& a, const Student& b) {
    return a.average > b.average;
}

int main() {
    int n;
    cout << "请输入学生人数: ";
    cin >> n;

    vector<Student> students(n);

    // 输入信息
    for (int i = 0; i < n; ++i) {
        cout << "正在输入第 " << i + 1 << " 个学生的信息: " << endl;
        cout << "学号: "; cin >> students[i].id;
        cout << "姓名: "; cin >> students[i].name;
        cout << "请输入8门课的成绩: ";
        for (int j = 0; j < 8; ++j) {
            cin >> students[i].scores[j];
        }
        students[i].calculateAverage(); // 计算平均分
    }

    // 按照平均成绩由高到低排序
    sort(students.begin(), students.end(), compareStudents);

    // 输出结果
    cout << "\n--- 学生成绩排名表 (按平均分降序) ---\n";
    cout << left << setw(15) << "学号" << setw(10) << "姓名";
    for (int j = 1; j ≤ 8; ++j) cout << "课" << j << " ";
    cout << "平均分" << endl;

```

```

for (const auto& s : students) {
    cout << left << setw(15) << s.id << setw(10) << s.name;
    for (int j = 0; j < 8; ++j) {
        cout << fixed << setprecision(1) << s.scores[j] << " ";
    }
    cout << " [" << s.average << "]" << endl;
}

return 0;
}

```

2.运行截图

```

Microsoft Visual Studio 调试控制  x
请输入学生人数: 2
正在输入第 1 个学生的信息:
学号: 1
姓名: 2
请输入8门课的成绩: 4 324 23523 12 123 123 13 123
正在输入第 2 个学生的信息:
学号: 4
姓名: 3
请输入8门课的成绩: 23 123 123 213 123 3 323 32

--- 学生成绩排名表 (按平均分降序) ---
学号      姓名      课1  课2  课3  课4  课5  课6  课7  课8  平均分
1          2          4.0 324.0 23523.0 12.0 123.0 123.0 13.0 123.0 [3030.6]
4          3          23.0 123.0 123.0 213.0 123.0 3.0 323.0 32.0 [120.4]

C:\Users\Wang Ziheng\source\repos\cpp_shuati\x64\Debug\cpp_shuati.exe (进程 47128)已退出, 代码为 0 (0x0)。
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口...

```